

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы № 5

по дисциплине «Программная инженерия»

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2026

Тема: разработка программного обеспечения ПС

Цель: приобретение навыков проектирования и разработки программного обеспечения ПС при помощи современных инструментальных средств.

Задание:

1. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, и решениями по выбору средств разработки специального ПО программной системы, принятыми при выполнении лабораторной работы 3, выбрать системные, служебные и прикладные программы, необходимые для разработки и эксплуатации ПС. Также определить укрупненные модули специального ПО. Разработать схему ПО программной системы на подобие схемы, показанной на рис. 6.
2. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, а также проектными решениями, разработанными при выполнении лабораторных работ 2 и 4, разработать специальное программное обеспечение ПС.
3. Выполнить описание разработанных компонентов приложения в виде таблицы. Типы компонентов указать согласно табл. 9 из методических указаний. Имена компонентов-файлов привести с указанием расширения.
4. Построить структурную схему разработанного приложения в виде диаграммы компонентов UML, выражающую взаимодействие его компонентов с компонентами БД в процессе функционирования приложения.
5. Запустить приложение на выполнение. Убедиться в соответствии результатов выполнения приложения требованиям, установленным в задании. При обнаружении логических ошибок техническом задокументировать их и устранить.
6. Представить экранные формы компонентов приложения, в том числе отчетов (при наличии).
7. Проанализировать код приложения по критерию сложности. В качестве критерия сложности использовать: число модулей (классов) приложения, суммарное число переменных подпрограмм (методов классов), включая их формальные параметры, суммарное количество операторов подпрограмм (методов классов), глубину вложенности структурных операторов ветвления и повторения, глубину наследования классов.
8. Выполнить описание физических элементов ПС в виде таблицы.
9. Построить диаграмму развертывания UML, выражающую зависимости между узлами ПС и развернутыми на них компонентами.

Ход выполнения работы:

1. Выбор общесистемного ПО и разработка схемы ПО ПС

Для развёртывания и эксплуатации ПС рассмотрены две современные серверные ОС (Таблица 1):

Характеристика	Windows Server 2022	Ubuntu Server 24.04 LTS
Тип лицензии	Платная	Бесплатная (Open Source)
Совместимость с VS Code	Отличная	Отличная
Поддержка Python	Хорошая	Отличная
Поддержка PostgreSQL	Есть	Есть
Удобство для разработки	Высокое	Среднее

Таблица 1. Анализ и выбор операционной системы

Выбрана Windows как операционная система, на которой ведётся разработка, и которая обеспечивает полную совместимость со всеми используемыми инструментами (VS Code, Python, PostgreSQL, браузер)

Для выполнения клиентской части ПС рассмотрены два современных браузера (Таблица 2):

Характеристика	Google Chrome	Mozilla Firefox
Инструменты разработчика	Отличные (DevTools)	Отличные
Поддержка ES6+	Полная	Полная
Поддержка Canvas API	Да	Да
Производительность JS	Высокая	Высокая

Таблица 2. Анализ и выбор браузера

Выбран Google Chrome как браузер с наиболее удобными инструментами разработчика для отладки клиентской части

Выбор инструментальных средств: на основе анализа, проведённого в ЛР №3, для разработки ПС используются:

- 1) IDE: Visual Studio Code
- 2) Языки: Python (сервер), JavaScript ES6+ (клиент)
- 3) СУБД: PostgreSQL (поддержка JSONB)
- 4) Система контроля версий: Git + GitHub

На основе анализа предметной области и требований технического задания определены следующие укрупнённые модули специального ПО, подлежащие реализации (Таблица 3):

Модуль	Назначение
Модуль парсинга	Преобразование строки в математическую функцию
Вычислительные модули	Решение уравнений, интегралов, ОДУ, СЛАУ
Модуль визуализации	Построение графиков и таблиц
Нейросетевая подсистема	Распознавание запросов, рекомендация методов
Серверный модуль	API на FastAPI
Модуль хранения	Взаимодействие с PostgreSQL

Таблица 3. Укрупнённые модули специального ПО

Разработанная структурная схема программного обеспечения ПС представлена на рисунке 1.

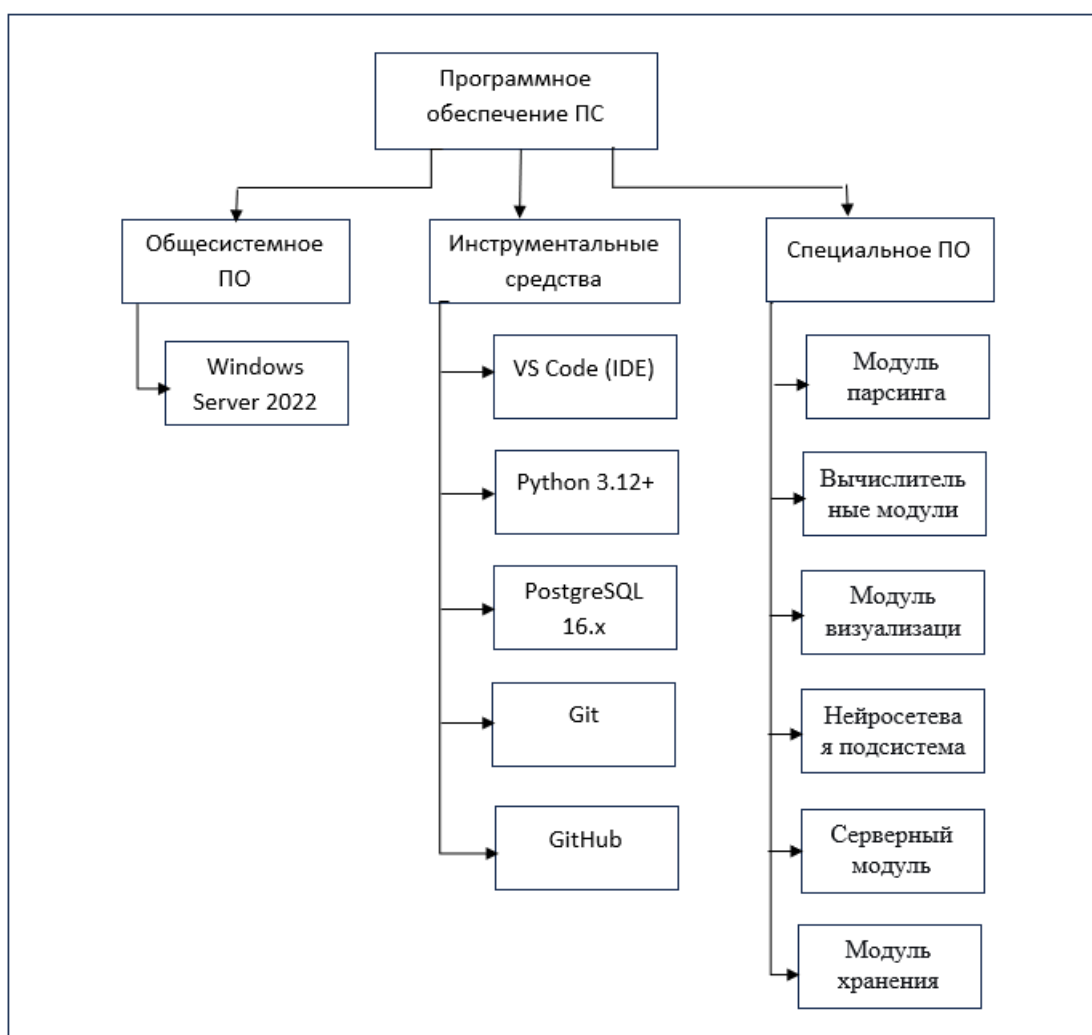


Рисунок 1. Структурная схема ПО ПС

Разработанная структурная схема полностью соответствует требованиям технического задания и отражает все компоненты, необходимые для разработки и эксплуатации интеллектуальной программной системы

2. Разработка специального программного обеспечения ПС

В соответствии с требованиями технического задания (ЛР №1) и проектными решениями, разработанными в ЛР №2 и ЛР №4, реализовано специальное программное обеспечение интеллектуальной системы «intelligent-math-solver».

Разработанное специальное ПО представляет собой клиент-серверное приложение с веб-интерфейсом. Клиентская часть реализована на JavaScript (ES6+) и HTML5/CSS3, серверная часть на Python с использованием фреймворка FastAPI. Хранение данных осуществляется в СУБД PostgreSQL.

В составе специального ПО реализованы следующие численные методы:

- 1) Для решения уравнений: методы Ньютона, секущих, половинного деления, простой итерации.
- 2) Для численного интегрирования: методы Симпсона, трапеций, прямоугольников (средних), Монте-Карло.
- 3) Для решения обыкновенных дифференциальных уравнений: методы Эйлера и Рунге-Кутты 4-го порядка.
- 4) Для решения систем линейных алгебраических уравнений: методы Гаусса, Якоби и Зейделя.

Реализована интеллектуальная подсистема, выполняющая:

- 1) распознавание типа задачи по текстовому запросу пользователя;
- 2) рекомендацию оптимального численного метода на основе истории решений, накопленной в PostgreSQL.

Разработан серверный модуль на FastAPI, обеспечивающий API для сохранения результатов решений и загрузки истории. Данные хранятся в таблице «training_data» СУБД PostgreSQL, содержащей поля: тип задачи, входные данные (JSONB), использованный метод, результат (JSONB).

Реализован модуль построения графиков и таблиц итераций на основе библиотеки Chart.js, обеспечивающий наглядное отображение процесса решения задач.

3. Описание разработанных компонентов приложения

Разработанное специальное ПО включает следующие компоненты (описание в формате табл. 8 согласно методическим указаниям):

№	Имя	Тип	Описание
1	index.html	<<form>>	Главная HTML-страница, структура вкладок
2	css/base.css	<<source>>	Базовые стили приложения
3	css/forms.css	<<source>>	Стили форм ввода
4	css/tables.css	<<source>>	Стили таблиц результатов
5	css/charts.css	<<source>>	Стили для графиков
6	js/core/main.js	<<source>>	Главный модуль, инициализация приложения
7	js/core/event-manager.js	<<source>>	Обработчик событий, связь UI с логикой
8	js/math-core/math-parser.js	<<source>>	Парсинг математических выражений
9	js/math-core/math-parserDE.js	<<source>>	Парсинг дифференциальных уравнений
10	js/math-core/system-parser.js	<<source>>	Парсинг систем уравнений
11	js/numerical-methods/equations/*.js	<<source>>	Методы решения уравнений (4 файла)
12	js/numerical-methods/integration/*.js	<<source>>	Методы интегрирования (4 файла)
13	js/numerical-methods/differential/*.js	<<source>>	Методы решения ОДУ (2 файла)
14	js/numerical-methods/systems/*.js	<<source>>	Методы решения СЛАУ (3 файла)
15	js/visualization/charts/chart-manager.js	<<source>>	Управление графиками
16	js/visualization/charts/chart-builder.js	<<source>>	Построение графиков
17	js/neural/neural-solver.js	<<source>>	Нейросетевая подсистема

18	js/neural/text-corrector.js	<<source>>	Исправление грамматических ошибок
19	js/neural/method-recommender.js	<<source>>	Рекомендация методов на основе истории
20	app.py	<<source>>	Серверный модуль (FastAPI)
21	init_db.py	<<source>>	Инициализация базы данных
22	База данных PostgreSQL	<<database>>	Хранение истории решений

Таблица 4. Перечень разработанных компонентов приложения

Взаимодействие компонентов: клиентская часть (браузер) загружает index.html и связанные CSS/JS файлы. При вводе данных пользователем event-manager.js вызывает соответствующий вычислительный модуль, который использует math-parser.js для парсинга выражений. Результат передаётся в chart-builder.js для визуализации и через серверный модуль app.py сохраняется в PostgreSQL. Нейросетевая подсистема анализирует текстовые запросы, исправляет ошибки и рекомендует методы на основе сохранённой истории

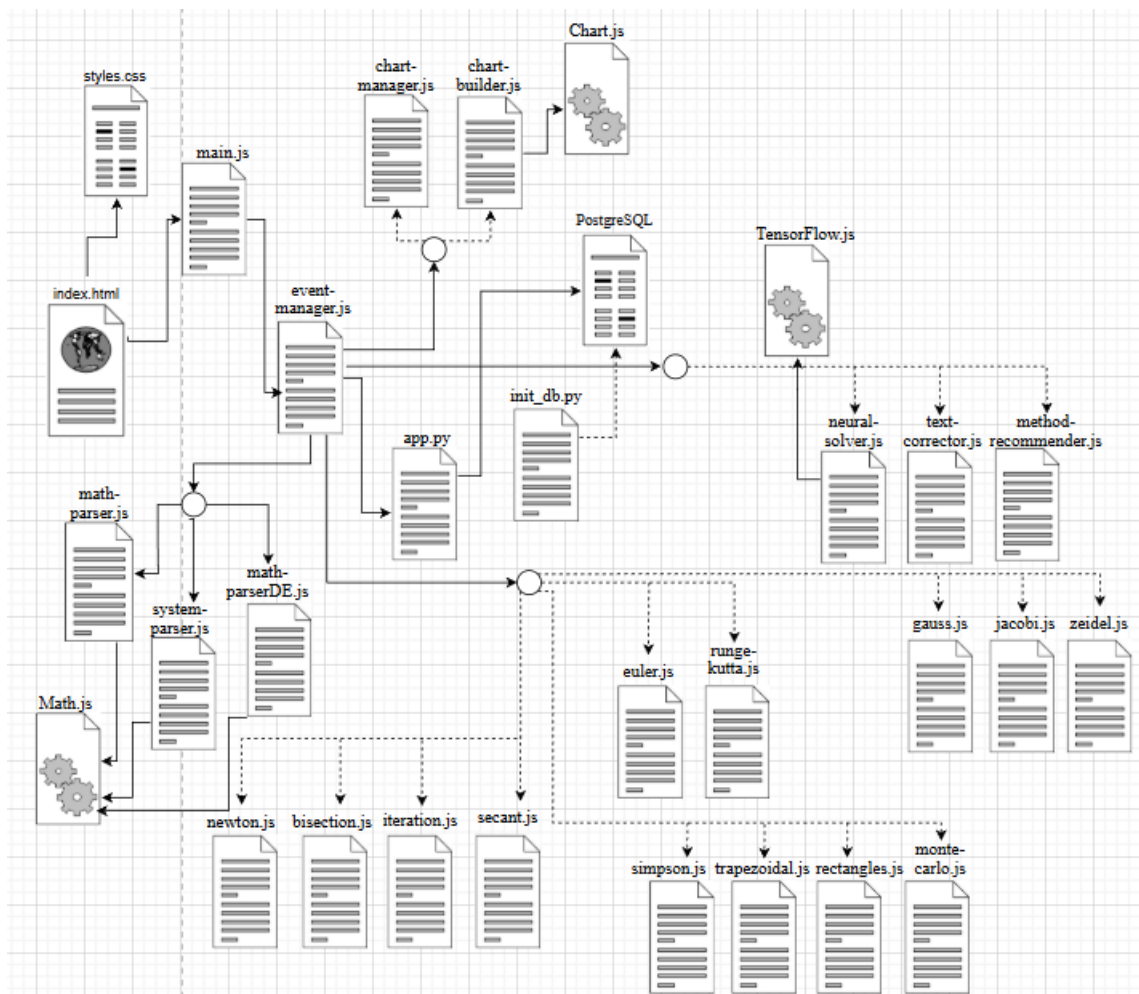


Рисунок 2. Диаграмма компонентов

Компонент index.html (веб-страница) через экспортируемый интерфейс инициализирует компонент main.js. Тот, в свою очередь, через запускает компонент event-manager.js, который является центральным обработчиком. Диаграмма компонентов показывает, что центральным управляющим компонентом является event-manager.js, который получает команды от index.html через main.js и распределяет задачи между вычислительными модулями (newton.js, bisection.js, simpson.js, euler.js, gauss.js и др.), парсерами (math-parser.js, system-parser.js, math-parserDE.js), модулем визуализации (chart-builder.js) и нейросетевой подсистемой (neural-solver.js). Парсеры используют библиотеку Math.js, модуль визуализации использует Chart.js, нейросетевая подсистема использует TensorFlow.js. Результаты вычислений через app.py сохраняются в PostgreSQL, а скрипт init_db.py служит для первоначального создания базы данных.

4. Тестирование специального ПО

1) Модульное тестирование

Проведено модульное тестирование ключевых компонентов специального программного обеспечения методом чёрного ящика. Тестирование выполнялось через консоль браузера с прямым вызовом методов.

Результаты модульного тестирования представлены на рисунке 3

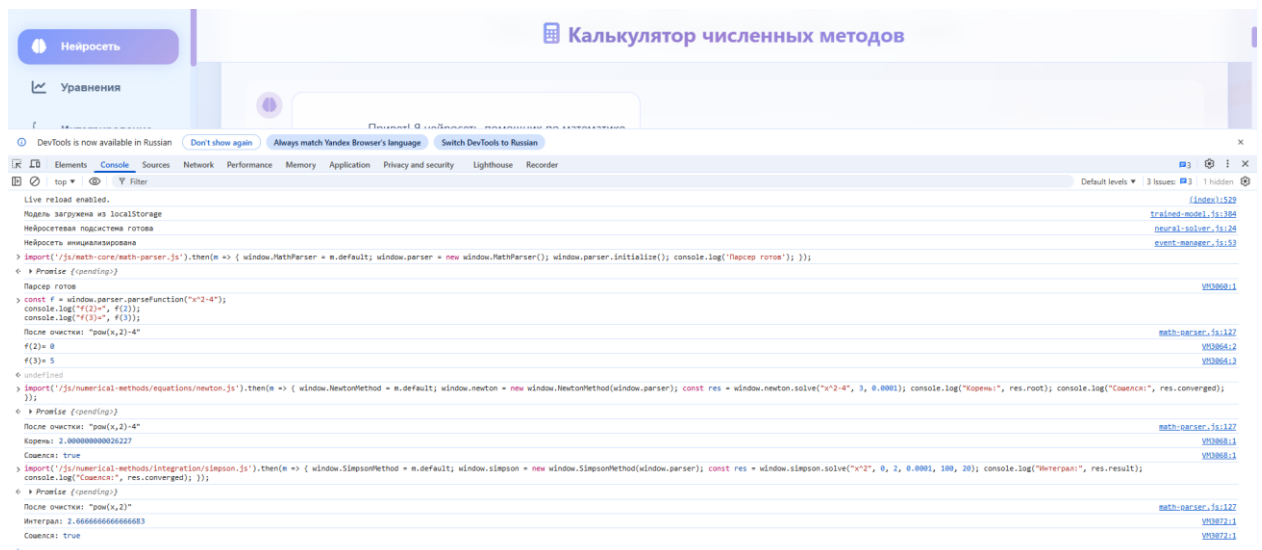


Рисунок 3. Результаты модульного тестирования в консоли браузера

В консоли видно, что функция $f(x) = x^2 - 4$ при подстановке $x=2$ даёт 0 (корень уравнения), а при $x=3$ даёт 5. Метод Ньютона нашёл корень $x=2$ с

высокой точностью. Метод Симпсона вычислил интеграл $\int x^2 dx$ от 0 до 2, получив значение 2.66666667 (это правильный ответ).

Все модульные тесты пройдены успешно. Парсер корректно вычисляет значения функций, численные методы сходятся к ожидаемым результатам с высокой точностью.

2) Системное тестирование

Проведено системное тестирование приложения через пользовательский интерфейс (вкладка «Нейросеть»). Тестирование выполнялось методом чёрного ящика.

На рисунке 4 показано, как программа решает простое уравнение

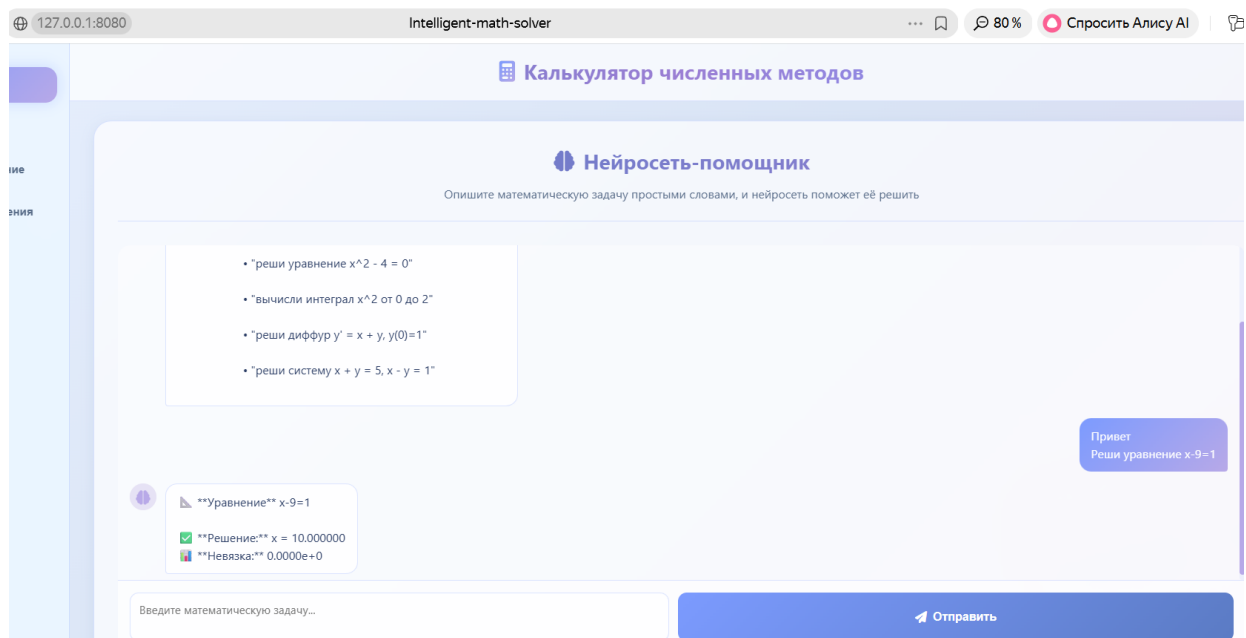


Рисунок 4. Решение уравнения

Пользователь вводит «решите уравнение $x-9=1$ ». Программа правильно понимает, что это уравнение, вычисляет корень и выводит ответ: $x=10$. Всё работает верно.

На рисунке 5 показано вычисление определённого интеграла

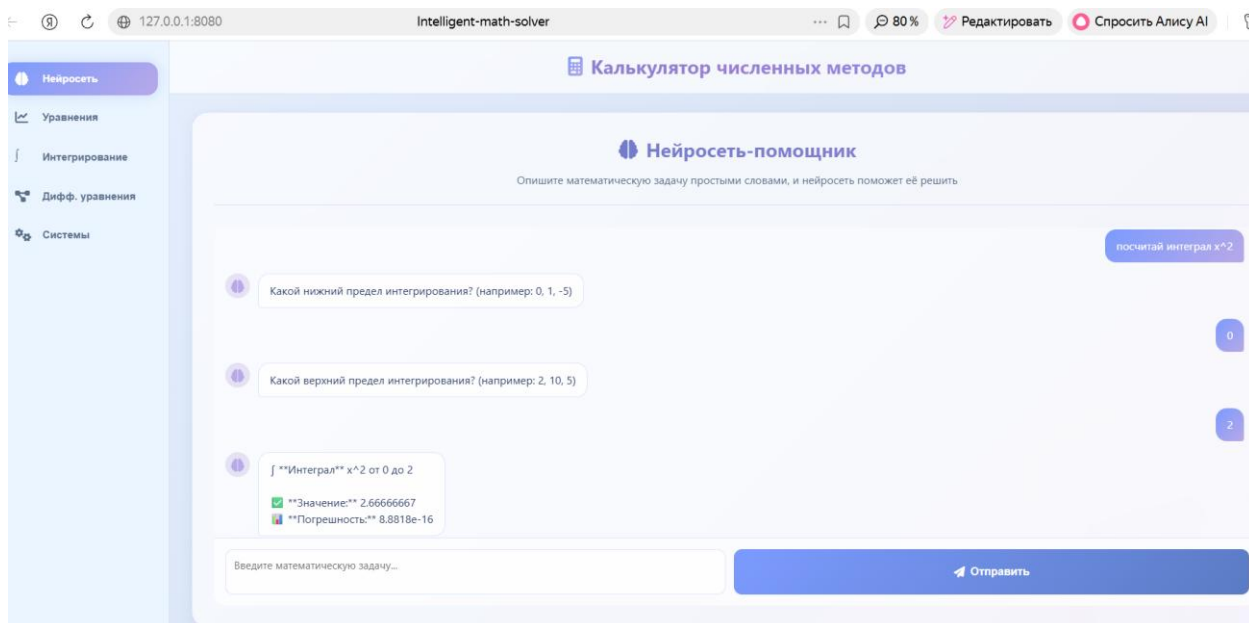


Рисунок 5. Вычисление интеграла

Пользователь просит вычислить интеграл x^2 от 0 до 2. Программа даёт ответ 2.66666667, это правильное значение интеграла. Также видно, что если не указать пределы интегрирования, программа сама спросит их у пользователя

На рисунке 6 показаны два способа решения ДУ

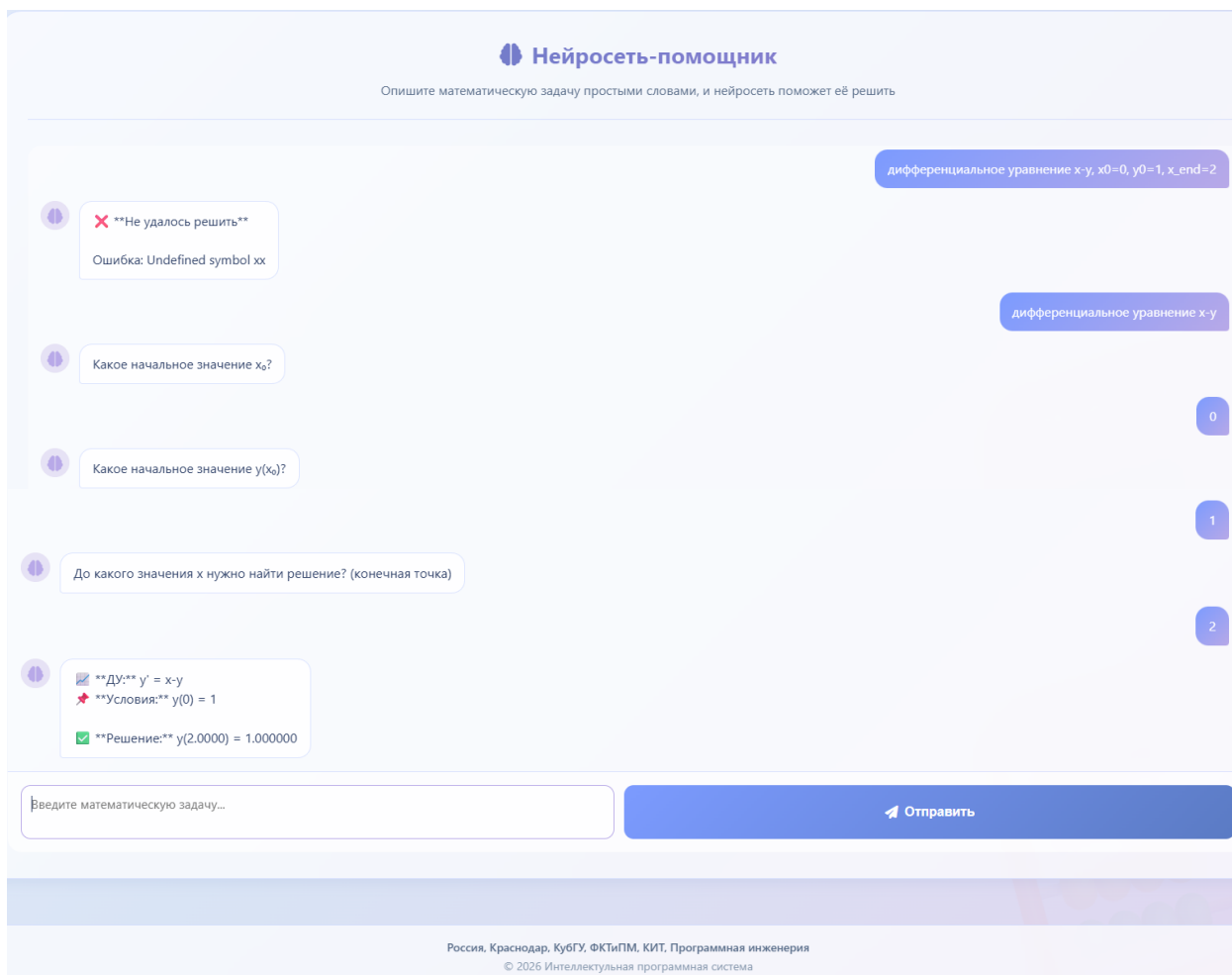


Рисунок 6. Решение дифференциальных уравнений (две формулировки)

В первом случае пользователь указал все параметры сразу: «дифференциальное уравнение $x-y$, $x_0=0$, $y_0=1$, $x_{\text{end}}=2$ ». Программа не смогла распознать запрос. Во втором случае пользователь не указал параметры, и программа задала уточняющие вопросы (x_0 , y_0 , конечная точка), после чего успешно решила уравнение

На рисунке 7 показано решение системы линейных уравнений

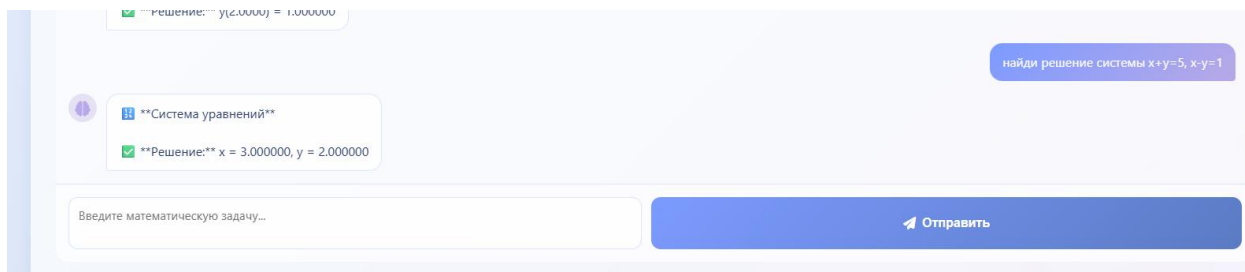


Рисунок 7. Решение СЛАУ

Пользователь вводит систему из двух уравнений. Программа находит решение: $x=3$, $y=2$. Это правильный ответ, система решена верно

На рисунке 8 показано, как программа реагирует на опечатки (грамматические ошибки) в запросе

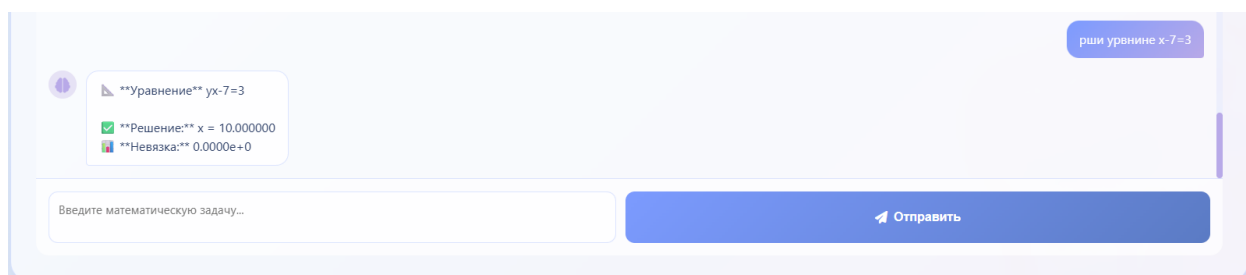


Рисунок 8. Запрос с опечаткой

Пользователь написал «рши урвнине» вместо «реши уравнение». Программа всё равно поняла и правильно решила уравнение, получив $x=10$

На рисунке 9 показано, как программа отвечает на запрос, не связанный с математикой

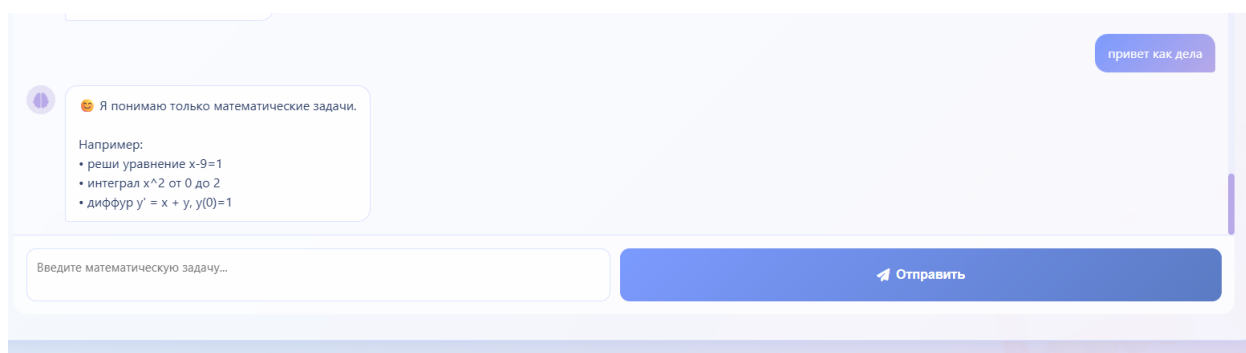


Рисунок 9. Обработка бессмысленного запроса

Пользователь написал простое приветствие. Программа не пытается решать его как уравнение, а сообщает, что понимает только математические задачи, и приводит примеры корректных запросов. Это правильное поведение, так как программа не должна ломаться на посторонних фразах

Выявленные несоответствия:

В процессе системного тестирования выявлено следующее несоответствие:

Баг-репорт №1

Короткое описание (Summary)	Ошибка парсинга дифференциального уравнения при передаче параметров в одной строке
Проект (Project)	intelligent-math-solver
Компонент приложения (Component)	Нейросетевая подсистема (chat-neural.js)

Номер версии (Version)	1.0
Шаги воспроизведения (Steps to Reproduce)	1. Открыть вкладку «Нейросеть» 2. Ввести запрос: «дифференциальное уравнение $x-y$, $x_0=0$, $y_0=1$, $x_{end}=2$ » 3. Нажать кнопку «Отправить»
Фактический Результат (Result)	В окне чата сообщение: «Ошибка: Undefined symbol xx»
Ожидаемый результат (Expected Result)	Система выдает решение $y(2) = 1$

Таблица 5. Баг-репорт

Исправление: Внесены изменения в методы `extractX0`, `extractY0`, `extractXEnd` класса `ChatNeural`, а также в метод `solveODEFromNeural` класса `EventManager`. После исправления запрос обрабатывается корректно (см. рисунок 6).

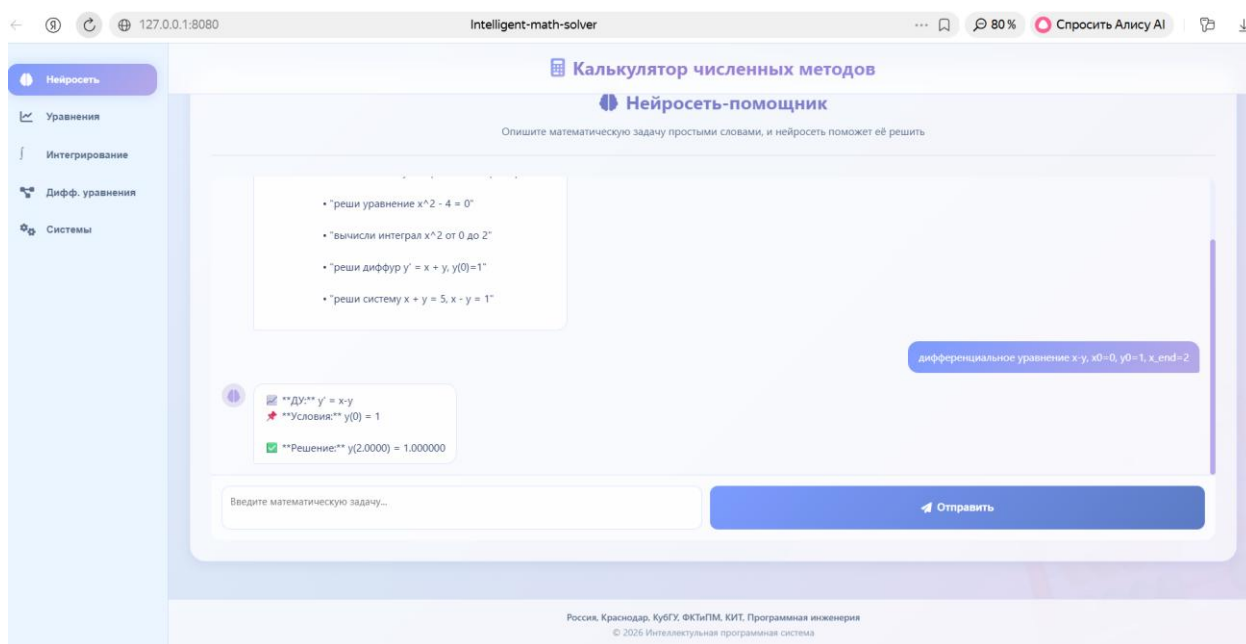


Рисунок 10. Исправленное выполнение

3) Анализ сложности кода

Был проведён анализ исходного кода приложения по критериям сложности, указанным в задании. Результаты представлены в таблице 6

Число модулей (классов)	25
Суммарное число переменных подпрограмм (включая формальные параметры)	~180

Суммарное количество операторов подпрограмм (методов классов)	~2450
Глубина вложенности структурных операторов ветвления и повторения	4
Глубина вложенности рекурсий	0
Глубина наследования классов	1

Таблица 6. Анализ сложности кода

Число модулей включает: 1 главный модуль (main.js), 1 обработчик событий (event-manager.js), 3 парсера, 13 вычислительных модулей (4 на уравнения, 4 на интегралы, 2 на ДУ, 3 на СЛАУ), 3 модуля визуализации, 4 модуля нейросети, 2 серверных модуля (app.py, init_db.py).

Суммарное число переменных получено приблизительным подсчётом: каждый метод класса в среднем содержит 5-10 переменных, всего в приложении около 25-30 методов.

Суммарное количество операторов оценено визуальным просмотром кода: средний размер одного JS-файла около 100 строк кода, 20 файлов примерно 2000 строк, плюс Python-файлы около 100 строк, итого ориентировочно 2100-2500 операторов.

Глубина вложенности встречается в методе parseEquation класса SystemParser, где есть вложенные циклы и условные операторы. В остальных методах глубина не превышает 2-3

Рекурсии отсутствуют. Это сделано намеренно, чтобы избежать переполнения стека при большом количестве итераций.

Наследование не используется, так как все классы решают узкоспециализированные задачи и не требуют общей иерархии. Каждый класс экспортируется независимо.

5. Описание физических элементов ПС и диаграмма развертывания

В соответствии с требованиями задания составлено описание физических узлов программной системы. Типы элементов указаны согласно таблице 11 методических указаний

№	Имя	Тип	Описание
1	Клиентский ПК	<<device>>	Компьютер пользователя с браузером Google Chrome
2	Сервер приложений	<<processor>>	Сервер с запущенным FastAPI (Python)
3	Сервер баз данных	<<database>>	Сервер с СУБД PostgreSQL
4	Локальная сеть	<<net>>	Сетевое соединение между клиентом и сервером

Таблица 7. Перечень узлов программной системы

На рисунке 14 представлена диаграмма развертывания, отражающая физическое размещение компонентов программной системы на узлах и связи между ними

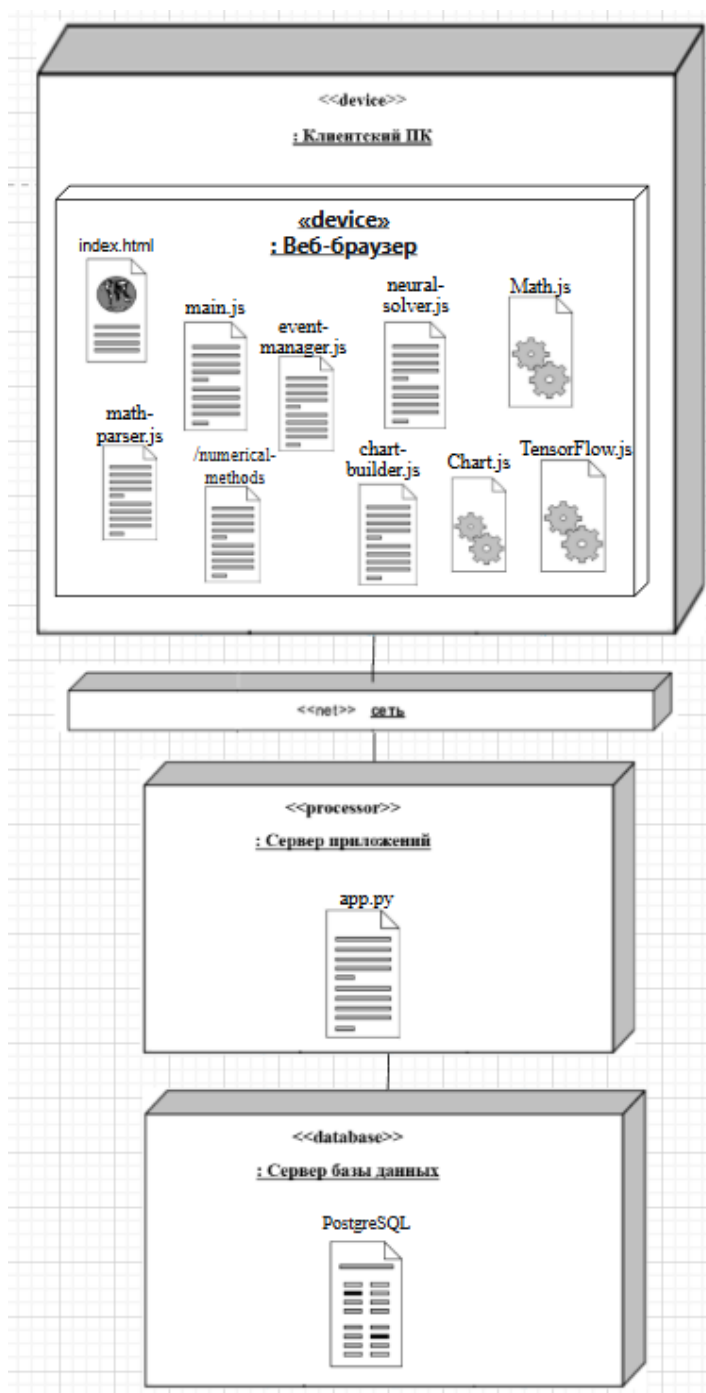


Рисунок 11. Диаграмма развертывания ПС

Клиентский ПК (<<device>>) - пользовательский компьютер, на котором открыт браузер. Браузер загружает HTML-страницу и все необходимые JavaScript-модули. Клиентская часть отвечает за отображение интерфейса, ввод данных, визуализацию графиков и нейросетевое распознавание запросов. Внешние библиотеки (Chart.js, TensorFlow.js, math.js) подключаются и исполняются также в браузере.

Сервер приложений (<<processor>>) - это узел, на котором запущен серверный модуль app.py на фреймворке FastAPI. Сервер обрабатывает два типа запросов:

- GET /api/history - получение истории решённых задач
- POST /api/save-result - сохранение нового результата в базу данных

Сервер БД (<<database>>) - узел с СУБД PostgreSQL, где хранится таблица training_data. Таблица содержит следующие поля: id (первичный ключ), task_type (тип задачи: equation, integral, ode, system), input_data (входные данные в формате JSONB), method_used (использованный метод), result (результат решения в формате JSONB).

Сеть (<<net>>) обеспечивает связь между узлами. Клиент взаимодействует с сервером приложений по протоколу HTTP. Сервер приложений взаимодействует с сервером БД по протоколу TCP

Вывод: в ходе выполнения лабораторной работы разработана интеллектуальная программная система «intelligent-math-solver», реализующая решение математических задач (уравнения, интегралы, ОДУ, СЛАУ) с использованием численных методов и нейросетевой подсистемы на базе TensorFlow.js. Проведённое модульное и системное тестирование подтвердило корректность работы всех компонентов, выявленные ошибки были устранены. Система полностью соответствует требованиям технического задания и готова к эксплуатации.